Audit de sécurité — BrainStore ERP

Périmètre : **Administration** (authentication, users, admin_rbac, system_config, core) & **Transport / Voyages** (voyages_config, voyages_trips, voyages_booking, transport_config). Date : 2026-10-13. Classé du plus critique au moins critique. Revue statique (lecture de code) ; aucune modification apportée.

Thème dominant. Le risque principal est l'**autorisation au niveau objet (multi-tenant)** dans le module Administration (RBAC & agences) : plusieurs endpoints se fient à un identifiant d'objet et/ou à un paramètre `company` fourni par le client, gardés seulement par « est admin d'institution », sans vérifier que l'objet appartient à la société de l'appelant. Le module Transport, lui, isole correctement par société.

Synthèse (par criticité)

#	Réf	Sévérité	Intitulé
1	A-1	CRITICAL	Attribution/révocation de rôles inter-tenant (Broken Object-Level Authorization)
2	A-2	HIGH	Lecture/modification/suppression des rôles & profils d'autres sociétés
3	A-3	HIGH	Accès inter-tenant aux agences via le paramètre « company » du client
4	A-4	HIGH	HasModulePermission échoue en mode « ouvert » (fail-open)
5	A-5	MEDIUM	2FA : le second facteur n'est pas lié à l'étape mot de passe
6	A-6	LOW	Divulgation de configuration RBAC d'autres sociétés via « company »
7	A-7	INFO	Listing public des plans (AllowAny)
8	A-8	INFO	Incohérence mineure IsInstitutionAdmin vs IsSuperAdmin

Détail des vulnérabilités

#1 CRITICAL A-1 — Attribution/révocation de rôles inter-tenant (Broken Object-Level Authorization)

EMPLACEMENT

- `apps/admin_rbac/views.py — UserRoleAssignView / UserRoleRevokeView / UserProfileAssignView / UserRoleListView / UserPermissionsView`
- `apps/admin_rbac/services/role_services.py — assign_role()`

DESCRIPTION

Les endpoints d'affectation de rôles/profils récupèrent l'utilisateur cible par `User.objects.get(id=user_id)` **SANS FILTRER PAR SOCIÉTÉ**, et prennent le `company_id` **DEPUIS LE CORPS DE LA REQUÊTE** (contrôlé par l'appelant). Le service `assign_role()` ne valide le couple rôle↔société que pour les rôles *non système* : un rôle système (`company_id = NULL`) contourne la garde. Rien ne vérifie que l'admin appelant possède bien le `company_id` fourni, ni que l'utilisateur cible appartient à sa société. Seule protection : `IsInstitutionAdmin` (= « est un admin d'une société quelconque »).

IMPACT

Un admin d'institution de la Société A peut attribuer ou révoquer des rôles (y compris des rôles système) à des utilisateurs d'autres sociétés, ou dans un `company_id` arbitraire — manipulation des droits inter-tenant. Atteinte directe au cloisonnement multi-sociétés.

CORRECTIF RECOMMANDÉ

Imposer, côté vue et côté service, que (1) l'utilisateur cible appartient à la société de l'appelant (sauf super-admin), (2) le `company_id` = société de l'appelant (ignorer la valeur fournie pour les non-super-admins), (3) le rôle (même système) est autorisé dans cette société. Reproduire le schéma `_scope()` de voyages_trips.

#2 HIGH A-2 — Lecture/modification/suppression des rôles & profils d'autres sociétés

EMPLACEMENT

- `apps/admin_rbac/views.py — RoleDetailView (GET/PATCH/DELETE), RoleSetPermissionsView, ProfileDetailView, ProfileSetRolesView`
- `apps/admin_rbac/selectors/role_selectors.py — get_role(role_id) (aucun filtre société)`

DESCRIPTION

`get_role(pk)` / `get_profile(pk)` récupèrent l'objet par identifiant **SANS FILTRE SOCIÉTÉ**. Les vues ne sont gardées que par `IsInstitutionAdmin`. Les rôles/profils *système* sont protégés en écriture, mais tout rôle/profil **PERSONNALISÉ D'UNE AUTRE SOCIÉTÉ** reste lisible, renommable, re-permissionnable et supprimable (si non assigné).

IMPACT

Un admin de la Société A peut lire la définition et altérer/supprimer les rôles personnalisés de la Société B (avec un identifiant de rôle), impactant le contrôle d'accès d'un autre tenant.

CORRECTIF RECOMMANDÉ

Scoper `get_role` / `get_profile` par la société de l'appelant (ou refuser si l'objet n'appartient ni à sa société ni au périmètre système autorisé). Retourner 404 sinon.

#3 HIGH A-3 — Accès inter-tenant aux agences via le paramètre « company » du client

EMPLACEMENT

- `apps/system_config/views.py — AgencyDetailView._get (GET/PATCH/DELETE), AgencyListView`

DESCRIPTION

La résolution utilise `request.query_params.get("company") or _get_company_id(request)` : le `company` fourni par le client est honoré **POUR TOUS LES RÔLES**. Or un admin d'institution **CONTOURNE** `HasModulePermission`. En passant `?company=<autre_société>` avec l'identifiant d'agence correspondant, `get_agency(pk, company_id)` renvoie l'agence de l'autre société.

IMPACT

Un admin de la Société A peut consulter/modifier/supprimer une agence de la Société B. À comparer au schéma correct de voyages_trips `_scope()` (seul le super-admin peut préciser `company_id`).

CORRECTIF RECOMMANDÉ

N'accepter `company` / `company_id` du client que pour le super-admin ; pour les autres, forcer la société de l'utilisateur. Centraliser via un helper `_scope` unique.

#4 HIGH A-4 — HasModulePermission échoue en mode « ouvert » (fail-open)

EMPLACEMENT

- `apps/core/permissions/__init__.py:54–55 (HasModulePermission.has_permission)`

DESCRIPTION

Quand la méthode HTTP n'a pas d'entrée dans `permission_map` (ni `required_permission`), la permission **EST ACCORDÉE** (`if not code: return True`). Toute vue utilisant ce garde qui oublie de mapper une méthode l'ouvre à **TOUT UTILISATEUR AUTHENTIFIÉ**.

IMPACT

Élévation de privilège latente : une seule méthode non mappée suffit à exposer une action. Les deux modules audités (Administration, Transport) mappent correctement leurs méthodes aujourd'hui, mais le défaut par défaut est dangereux et non détecté à la revue.

CORRECTIF RECOMMANDÉ

Inverser le défaut en **DENY** : si aucun code n'est défini pour la méthode, refuser (ou exiger explicitement un marqueur `public = True`). Ajouter un test garantissant que toute vue `HasModulePermission` mappe chaque méthode implémentée.

#5 MEDIUM A-5 — 2FA : le second facteur n'est pas lié à l'étape mot de passe

EMPLACEMENT

- `apps/authentication/views.py — Login2FAVerifyView`

DESCRIPTION

L'étape 2 délivre les jetons à partir du seul `user_id` + un code TOTP valide, sans preuve que l'étape mot de passe a réussi (pas de jeton intermédiaire court). Les deux facteurs ne sont pas chaînés.

IMPACT

Quiconque possède le secret TOTP (et un `user_id`) peut s'authentifier **SANS LE MOT DE PASSE** : la 2FA se réduit à un facteur unique (TOTP) pour les comptes concernés.

CORRECTIF RECOMMANDÉ

À l'étape 1, émettre un jeton intermédiaire signé et à courte durée (lié au compte, usage unique) exigé à l'étape 2 ; l'étape 2 ne doit pas accepter un simple `user_id`.

#6 LOW A-6 — Divulgation de configuration RBAC d'autres sociétés via « company »

EMPLACEMENT

- `apps/admin_rbac/views.py — UserRoleListView, UserPermissionsView, RoleListView/ProfileListView (?company=)`

DESCRIPTION

Même cause racine que A-1/A-2/A-3 : `company` lu depuis la requête sans contrôle de propriété. En lecture, cela permet d'énumérer rôles/permissions/affectations d'une autre société.

IMPACT

Fuite d'informations sur le contrôle d'accès d'autres tenants (noms de rôles, permissions, affectations).

CORRECTIF RECOMMANDÉ

Même correctif que A-1/A-3 : forcer la société de l'appelant pour les non-super-admins.

#7 INFO A-7 — Listing public des plans (AllowAny)

EMPLACEMENT

- `apps/system_config/views.py — PlanListView / PlanDetailView (permission_classes = [AllowAny])`

DESCRIPTION

Les plans tarifaires sont lisibles sans authentification (probablement volontaire : page tarifs publique).

IMPACT

Faible : exposition des définitions de plans/tarifs. À confirmer comme intentionnel.

CORRECTIF RECOMMANDÉ

Confirmer le caractère public ; sinon restreindre aux utilisateurs authentifiés.

#8 INFO A-8 — Incohérence mineure IsInstitutionAdmin vs IsSuperAdmin

EMPLACEMENT

- `apps/core/permissions/__init__.py:26 (IsInstitutionAdmin vérifie is_superuser sans is_staff)`

DESCRIPTION

IsInstitutionAdmin accorde l'accès si `is_superuser` seul ; IsSuperAdmin exige `is_staff AND is_superuser`.

IMPACT

Négligeable isolément (un superuser non-staff reste de confiance). Harmoniser pour la cohérence.

CORRECTIF RECOMMANDÉ

Aligner la définition de « super-admin » (is_staff AND is_superuser) partout.

Points vérifiés — conformes

- Transport (voyages_trips/booking/config) : isolation multi-sociétés solide — _scope() force la société (seul le super-admin précise company_id) et les sélecteurs filtrent par société (get_trip(company, id)).
- Aucune injection : pas de .raw()/.extra()/RawSQL, eval/exec, os.system/subprocess, pickle/yaml.load dans les modules audités.
- Aucun serializer fields="__all__" ; la PATCH de profil (MeView) ne permet pas l'escalade (is_superuser/is_institution_admin en read_only).
- Création d'utilisateur : impossible de définir is_superuser ; société cible forcée à celle de l'admin.
- Authentification : verrouillage après N tentatives (lockout) présent ; endpoints companies/paramètres/feature-flags correctement réservés au super-admin.

Recommandation prioritaire

Introduire un helper d'autorisation d'objet unique (à l'image de `_scope()` de voyages_trips) et l'appliquer à **tout** endpoint d'Administration manipulant un objet par identifiant : (1) refuser un `company` client pour les non-super-admins, (2) vérifier l'appartenance de l'objet à la société de l'appelant, (3) renvoyer 404 sinon. Traiter A-1 → A-4 en priorité, puis durcir le défaut *fail-open* de `HasModulePermission`.